

TRABALHO PARA A DISCIPLINA DE TÉCNICAS DE PROGRAMAÇÃO – TURMAS S71 e S73 - DOS CURSOS DE EC & BSI DA UTFPR Curitiba: Shinobi Saga

Maria Eduarda de Gracia, Rachel Paulino Previtali
gracia@alunos.utfpr.edu.br, rachelprevitali@alunos.utfpr.edu.br

Disciplina: **Técnicas de Programação – CSE20 / S71** – Prof. Dr. Jean M. Simão
Departamento Acadêmico de Informática – DAINF - Campus de Curitiba
Curso Bacharelado em: Engenharia da Computação (EC) / Sistemas de Informação (BSI)
Universidade Tecnológica Federal do Paraná - UTFPR
Avenida Sete de Setembro, 3165 - Curitiba/PR, Brasil - CEP 80230-901

Resumo —A disciplina de Técnicas de Programação exige o desenvolvimento de um *software* de plataforma, no formato de um jogo, para fins de aprendizado de técnicas de engenharia de *software*, particularmente de programação orientada a objetos em C++. Para tal, neste trabalho, escolheu-se o jogo Shinobi Saga, no qual o jogador enfrenta inimigos em um **cenário japonês**. O jogo tem **duas fases** que se diferenciam por cenário e por dificuldades para o jogador. Para o desenvolvimento do jogo foram considerados os requisitos textualmente propostos e elaborada modelagem (análise e projeto) via Diagrama de Classes em Linguagem de Modelagem Unificada (*Unified Modeling Language - UML*) usando como base um diagrama assaz genérico e prévio proposto. Subsequentemente, em linguagem de programação C++ 2003, realizou-se o desenvolvimento que contemplou os conceitos usuais de Orientação a Objetos como Classe, Objeto e Relacionamento, bem como alguns conceitos ditos avançados como Classe Abstrata, Herança, Polimorfismo, Gabaritos, Persistências de Objetos por Arquivos, Sobrecarga de Operadores e Biblioteca Padrão de Gabaritos (*Standard Template Library - STL*). Depois da implementação, os testes e uso do jogo feitos pelos próprios desenvolvedores demonstraram sua funcionalidade conforme os requisitos e a modelagem elaborada. Por fim, salienta-se que o desenvolvimento em questão permitiu cumprir o objetivo de aprendizado visado.

Expressões-chave: Artigo-Relatório para o Trabalho em Técnicas de Programação, Trabalho Acadêmico Voltado a Implementação em C++ 2003, Elaboração de Jogo de Plataforma 2D como Projeto Acadêmico.

INTRODUÇÃO

O trabalho em questão, contextualizado na disciplina Técnicas de Programação, trata-se do desenvolvimento de um jogo de plataforma 2D. Este tem por objetivo aprimorar as habilidades dos estudantes nas áreas de codificação, além de lógica computacional, estruturação de projetos maiores e também da capacidade de trabalhar em equipe. Através desse trabalho, foi possível implementar diversos conceitos estudados em sala de aula.

Quanto ao objeto de estudo do trabalho, um jogo de plataforma 2D consiste em guiar um personagem através de um cenário bidimensional constituído por obstáculos e plataformas. Ademais, cabe ao jogador superar inimigos para enfim chegar ao final da fase ou, de forma alternativa, concluir algum outro objetivo no decorrer dela.

Em relação à implementação do trabalho, foi utilizada a linguagem de programação C++ orientada a objetos, baseando-se no método de Engenharia de Software. Inicialmente, realizou-se a compreensão dos requisitos propostos. Posterior a isso, procedeu-se a modelagem (análise e projeto) através de um diagrama de classes em UML, o qual foi derivado e expandido a partir de um modelo dado usando a ferramenta Astah UML. Já a etapa de implementação foi feita utilizando a linguagem já dita anteriormente e, finalmente, foram feitos testes do software.

O restante deste documento está organizado de forma a detalhar todo o desenvolvimento do projeto. A seção seguinte dedica-se à explicação do jogo em si, apresentando sua mecânica, funcionamento e imagens ilustrativas do jogo. Posteriormente, discute-se o desenvolvimento técnico, abordando os requisitos funcionais, suas implementações e a modelagem via diagrama UML. Posteriormente, tem-se a apresentação da tabela de conceitos utilizados e não utilizados e, por fim, são apresentadas as conclusões e a divisão de responsabilidades entre os membros da dupla.

EXPLICAÇÃO DO JOGO EM SI

O jogo desenvolvido, denominado “Shinobi Saga”, tem seu universo inspirado na cultura e no folclore japonês. O objetivo do jogador é guiar o personagem protagonizado por um shinobi através das fases. Sucintamente, shinobis eram guerreiros especialistas em espionagem, sabotagem e infiltração, atuando de forma independente durante o período feudal no Japão. Durante o jogo, faz-se necessário neutralizar (ou evitar) inimigos, além de ter que superar obstáculos que são encontrados durante o caminho para chegar ao final. O jogo possui duas fases e pode ser jogado tanto solo como em dupla.

A tela inicial, apresentada na figura 1, fornece algumas opções para o usuário: seleção da fase, carregamento do último jogo salvo e, ainda, visualização do ranking dos jogadores, onde é possível ver as cinco maiores pontuações registradas até aquele momento. Cabe mencionar que os pontos são obtidos ao derrotar inimigos durante o jogo.

Após selecionar a fase, é permitido ao usuário escolher entre o modo singleplayer ou multiplayer, e assim, inicia-se o jogo. Ressalta-se que, caso escolha-se começar da primeira fase, ao chegar em seu fim, é possível continuar a jogatina, seguindo para a segunda fase.



Figura 1. Tela inicial do jogo Shinobi Saga.

A figura 2 mostra uma visão geral da fase 1. No canto superior esquerdo da tela, tem-se a vida do jogador (em verde) e sua pontuação logo abaixo.

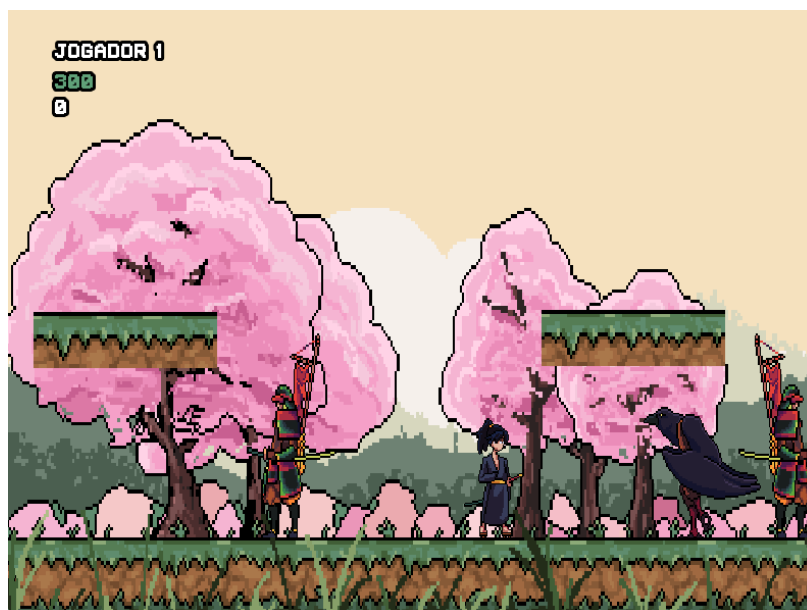


Figura 2. Panorama geral da fase 1.

A primeira fase do jogo se passa em um cenário rural japonês, onde o jogador irá encontrar Samurais e Tengus como inimigos, além de Plataformas e Redemoinhos como obstáculos.

A fim de contextualizar, Tengus são criaturas sobrenaturais do folclore japonês, retratados como seres traiçoeiros e associados a montanhas e florestas. Enquanto isso, Samurais foram guerreiros pertencentes à classe militar e que existiram durante o Japão feudal.

Os Samurais são os inimigos fáceis e a sua mecânica simples reside no comportamento de perambular de um lado para o outro, danificando e empurrando o jogador para longe caso haja uma colisão entre ambos. Um atributo exclusivo desse inimigo é a sua resistência, um valor que afeta diretamente no dano que o jogador consegue infligir no Samurai. Quanto maior a resistência, menor o dano causado nesse inimigo. Além disso, ao haver colisão entre os dois personagens em questão, o atributo “nível de maldade” aumenta em uma unidade, gerando um consequente aumento no dano causado pelas colisões posteriores. O cálculo do dano causado e o dano sofrido pelo Samurai é ilustrado nas seguintes equações:

$$Dano\ causado_{Samurai} = (nível\ de\ maldade_{Samurai}) \times 10$$

$$Dano\ sofrido_{Samurai} = (Dano\ causado_{Jogador}) \div (Resistência_{Samurai})$$



Figura 3. Inimigo Samurai (à esquerda) e jogador (à direita).

Já o segundo inimigo, que aparece tanto nesta quanto na segunda fase, é o Tengu. Esse inimigo, considerado de dificuldade média, possui um comportamento um pouco diferente do Samurai. Enquanto o jogador se mantiver fora de uma determinada área de “ativação” do Tengu, este inimigo irá perambular assim como o Samurai. Porém, ao atingir essa região, o Tengu entrará em um modo de perseguição, se aproximando do jogador e dando uma investida contra ele ao chegar perto o suficiente. Tal ato danifica e empurra o jogador. Ademais, a cada vez que o jogador ataca o Tengu, este tem seu nível de maldade incrementado. Isso faz aumentar o dano causado no jogador caso ambos se choquem novamente, seja pela investida ou devido a uma colisão qualquer entre os dois. As equações a seguir mostram o dano causado e recebido pelo Tengu, respectivamente:

$$Dano\ causado_{Tengu} = (nível\ de\ maldade) \times 50$$

$$Dano\ sofrido_{Tengu} = Dano\ causado_{Jogador}$$



Figura 4. Tengu perambulando.



Figura 5. Tengu no modo de perseguição.

Quanto aos obstáculos presentes nessa primeira fase, tem-se plataformas e redemoinhos. As plataformas, obstáculos fáceis e também presentes na segunda fase, permitem que o jogador se cure ao subir em cima delas pela primeira vez, além de ser possível usá-las para evitar inimigos. Das figuras 6 e 7, nota-se o aumento de 25 pontos de vida depois do jogador subir na Plataforma.

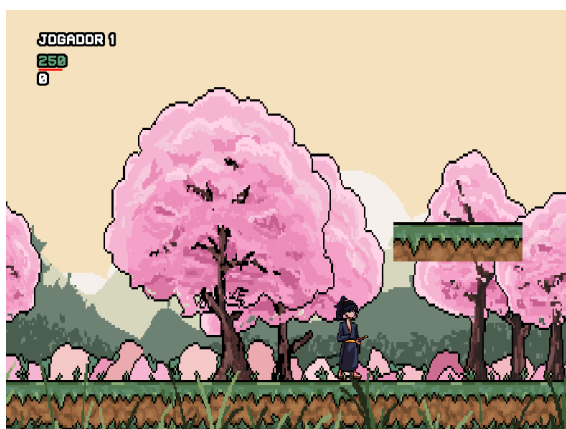


Figura 6. Jogador abaixo da Plataforma

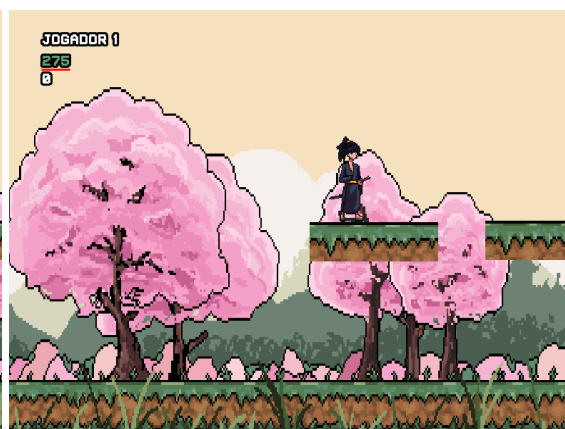


Figura 7. Jogador em cima da Plataforma

Já o obstáculo médio, denominado Redemoinho, exclusivo dessa fase 1, possui uma dinâmica de ativação e desativação, visto que, quando ativado, causa dano ao jogador caso se toquem. Caso contrário, o jogador não recebe dano do Redemoinho e consegue passar através dele durante o tempo que está desativado. As figuras 8 e 9 ilustram o comportamento desse obstáculo.



Figura 8. Redemoinho ativo.



Figura 9. Redemoinho desativado.

Já a fase 2 é ambientada em uma floresta escura, onde os inimigos encontrados são Kitsunes e Tengus e os obstáculos são as Plataformas e as Armadilhas de Urso.



Figura 10. Panorama geral da fase 2.

As kitsunes, conhecidas como raposa de nove caudas, também fazem parte da cultura folclórica japonesa. De forma breve, elas são seres míticos que possuem poderes mágicos e também podem assumir uma forma humana, assim como é representado visualmente na segunda fase do jogo.

Essa criatura lendária, considerada o “chefão” (inimigo difícil) do jogo, tem o seguinte comportamento: assim como o inimigo Tengu já descrito, a Kitsune caminha de um lado para o outro enquanto não houver um jogador dentro da sua região de “ativação”. Ao adentrar essa área, esse inimigo começará a lançar bolas de fogo na direção do jogador, que lhe causarão dano em caso de contato. Não só isso, mas caso o jogador colida corpo a corpo com a Kitsune, também tomará dano e será empurrado para longe.

Vale ressaltar também que, a cada vez que o jogador é atingido pela bola de fogo, o atributo “nível de maldade” da Kitsune é adicionado em um, aumentando o dano posterior causado pelo projétil.



Figura 11. Kitsune preparando o disparo.



Figura 12. Bola de fogo indo em direção ao jogador.

As equações a seguir mostram o dano causado e sofrido pela Kitsune:

$$\begin{aligned}
 \text{Dano}_{\text{projétil}} &= (\text{nível de maldade}) \times 20 \\
 \text{Dano causado (corpo a corpo)}_{\text{Kitsune}} &= 200 \\
 \text{Dano sofrido}_{\text{Kitsune}} &= \text{Dano causado}_{\text{Jogador}}
 \end{aligned}$$

O obstáculo difícil, exclusivo dessa fase é a Armadilha de Urso, danifica e atordoa o jogador por alguns segundos caso tente passar por ele sem saltar.



Figura 13. Armadilha de Urso à direita do jogador.



Figura 14. Jogador atordoado após sofrer dano.

DESENVOLVIMENTO DO JOGO

Esta seção irá contar o desenvolvimento mais técnico do jogo, onde primeiramente serão listados em formato de tabela os requisitos funcionais, junto de seu status (feito,

parcialmente feito ou não feito) e indicação de forma sucinta em quais classes ou conjunto de classes (namespaces ou pacotes) foi realizado tal requisito.

Tabela 1. Lista de Requisitos do Jogo de Plataforma e exemplos de Situações.

N.	Requisitos Funcionais	Situação	Implementação
1	Apresentar graficamente menu de opções aos usuários do Jogo, no qual pode se escolher fases, escolher ver colocação (<i>ranking</i>) de jogadores e escolher demais opções pertinentes (previstas nos demais requisitos).	REALIZADO	Cf. classes herdadas de Menu, como Menu_Principal.
2	Permitir um ou dois jogadores com representação gráfica aos usuários do Jogo, sendo que no último caso é para que os dois joguem de maneira concomitante.	REALIZADO	Cf. classe Jogador, cujos objetos (tanto o primeiro como o segundo jogador) são criados dinamicamente em Jogo.
3	Disponibilizar ao menos duas fases <u>distintas</u> , com chão, que podem ser jogadas sequencialmente ou selecionadas, via menu, nas quais jogadores tentam neutralizar inimigos por meio de algum artifício e vice-versa.	REALIZADO	Cf. pacote Fases, que contém as classes Fase, Fase_Primeira e Fase_Segunda. A fase 2 pode ser jogada sequencialmente caso o jogador chegue ao final da extensão da fase 1. Cada fase pode ser jogada independente da outra, via seleção no menu.
4	Ter pelo menos três tipos distintos de inimigos, cada qual com sua representação gráfica, sendo que ao menos um deles deve poder lançar projétil contra o(s) jogador(es) e um dos inimigos deve ser um ‘chefão’.	REALIZADO	Cf. pacote Personagens, que contém as classes Samurai_Inimigo, Tengu e Kitsune (dentre outras classes). Kitsune é o inimigo ‘chefão’ que lança projéteis contra o(s) jogador(es).
5	Ter a cada fase ao menos dois tipos de inimigos (<u>um deles exclusivo nela</u>) com número aleatório de instâncias, podendo ser várias instâncias (<u>definindo um máximo – e.g. dez</u>) e sendo pelo menos 3 instâncias <u>para cada tipo que estiver na fase</u> .	REALIZADO	O número aleatório foi definido usando a função rand(). Inimigos gerados na fase 1: Samurais e Tengus (sendo Samurai exclusivo desta fase). Inimigos gerados na fase 2: Tengus e Kitsunes (sendo Kitsune exclusivo desta fase).
6	Ter três tipos de obstáculos, cada qual com sua representação gráfica, sendo que ao menos um causa dano em jogador se eles colidirem.	REALIZADO	Cf. pacote Obstaculos, que contém as classes Plataforma, Redemoinho e Armadilha_de_Urso. Tanto o Redemoinho quanto a Armadilha

			causam dano quando há colisão com o jogador.
7	Ter em cada fase ao menos dois tipos de obstáculos (<u>um deles exclusivo nela</u>) com número aleatório (<u>definindo um máximo</u>) de instâncias (<i>i.e.</i> , objetos), sendo pelo menos 3 instâncias por tipo.	REALIZADO	O número aleatório foi definido usando a função rand(). Obstáculos gerados na fase 1: Redemoinhos e plataformas (sendo Redemoinho exclusivo desta fase). Obstáculos gerados na fase 2: Armadilhas de urso e plataformas (sendo a Armadilha exclusiva desta fase).
8	Ter em cada fase um cenário de jogo constituído por obstáculos, sendo que parte deles devem ser plataformas ou similares, sobre as quais pode haver inimigos e podem subir jogadores. Em cada fase, só pode ter um tipo coincidente de imigo e um tipo coincidente de obstáculo (que é a plataforma) em relação as demais fases.	REALIZADO	Cf. pacote Fases, Obstáculos e Personagens.
9	Gerenciar colisões entre jogador para com os inimigos e seu conjunto de projeteis, bem como entre jogador para com os obstáculos. Ainda, todos eles devem sofrer o efeito de alguma 'gravidade' no âmbito deste jogo de plataforma vertical e 2D.	REALIZADO	Cf. pacote Gerenciadores, pacote Listas e pacote Fases.
10	Permitir: (1) salvar nome do usuário, manter/salvar pontuação (incrementada via neutralização de inimigos) do jogador controlado pelo usuário e gerar lista de pontuação (<i>ranking</i>). E (2) Pausar e Salvar/Recuperar Jogada.	REALIZADO	Cf. classes herdadas de Menu, como Menu_Salvamento_Ranking, Menu_Principal e Menu_Pause
Total de requisitos funcionais apropriadamente realizados. (Cada tópico realizado efetivamente vale 10%)			100% (cem por cento).
Os requisitos dependem em algo uns dos outros, na chamada interdependência de requisitos.			

O diagrama UML do jogo está representado na imagem 15, mostrando todos os pacotes com suas classes e relações pertinentes.

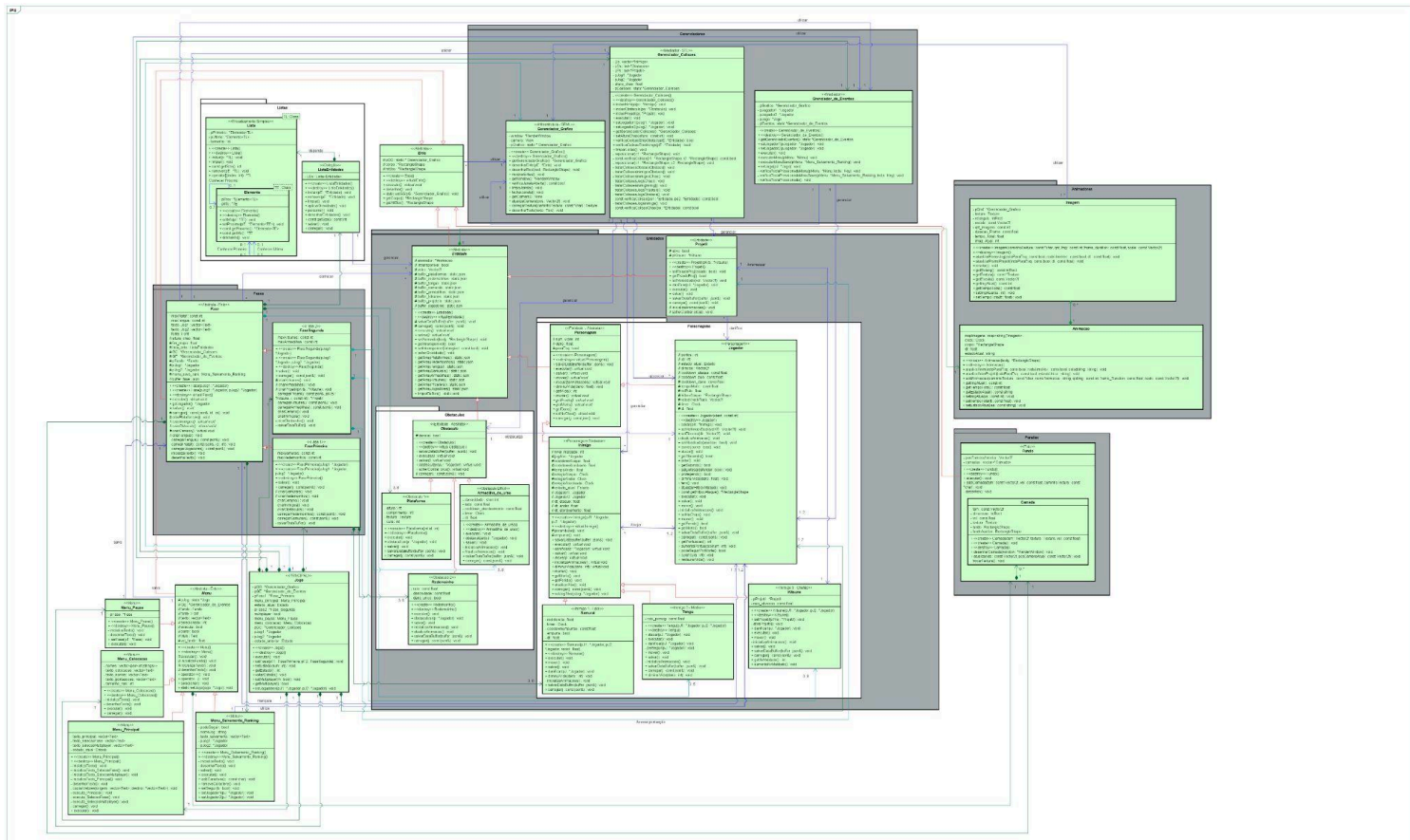


Figura 15. Diagrama de Classes do jogo em UML

A partir do diagrama acima, o jogo está modelado, de maneira resumida, da seguinte forma:

- A classe principal Jogo instancia o(s) jogador(es) e a fase, de acordo com a escolha do usuário no menu interativo da classe Menu, que usa o método de salvamento utilizando JSON. Além disso, é em Jogo que são criados todos os gerenciadores do pacote Gerenciadores;
- Classe abstrata Ente, na qual derivam as classes Entidade, Fase, Menu e Fundo, possuindo associação via ponteiro com o gerenciador gráfico para que todos os objetos derivados desta classe possam ser desenhados e visualizados.
- O namespace Gerenciadores inclui o Gerenciador Gráfico (que renderiza as janelas e as sprites), o Gerenciador de Eventos (que captura os inputs do teclado) e o Gerenciador de Colisões (verifica colisões entre as entidades);
- O namespace Fases contém a classe base Fase, Fase Primeira e Fase Segunda. A Fase Primeira e Fase Segunda, ao serem criadas em Jogo, instanciam os inimigos, os obstáculos e criam o cenário da fase. A classe base Fase se associa via ponteiro com os gerenciadores de colisão e de eventos, além de possuir uma lista de entidades, que é preenchida no momento de criação das entidades (inimigos e obstáculos);

- O namespace Listas, que possui a classe template Lista, contendo a classe aninhada (Nested) Elemento, ambas sendo usadas para gerenciamento de listas encadeadas de tipo genérico, e a classe do tipo coleção ListaEntidades, que possui uma lista encadeada parametrizada com Entidades .
- O namespace Entidades, na qual contém a classe abstrata Entidade que tem como herdeiras as classes Personagem, Obstaculo e Projtil. É englobado dentro do namespace Entidades, o namespace Personagens, que contém a classe abstrata Personagem da qual é derivada dela Jogador e a classe abstrata Inimigo, que, por sua vez, implementa por herança Samurai, Tengu e Kitsune. Por último, o namespace Entidades ainda engloba o namespace Obstaculos, que possui a classe abstrata Obstaculo, que implementa por herança as classes Plataforma, Redemoinho e Armadilha de Urso;
- O namespace Animadores, que contém as classes Imagem e Animacao, de forma geral, cuidam da animação das sprites dos jogadores, inimigos, obstáculos e projéteis. Enquanto a imagem trata de percorrer uma spritesheet, imagem com vários quadros de animação, a animação fica responsável por trocar de spritesheet, dependendo do estado da entidade que é recebido como parâmetro.
- O namespace Parallax contém as classes Fundo e Camada. O pacote em questão é responsável por produzir o efeito “parallax”, ou seja, da movimentação das várias camadas que compõem o fundo do cenário das fases.

Esta modelagem seguiu o Diagrama de Classes proposto nos requisitos do trabalho, expandindo-o, com as seguintes implementações:

- Criação da classe Animacao e da classe Imagem, para fazer as animações das entidades do jogo.
- Criação da classe Fundo e Camada, para produzir o efeito “parallax” nos cenários das fases.
- Criação de classes herdadas de Menu, para especializá-lo segundo o estágio do Jogo.

Por fim, destaca-se a utilização de conhecimentos interdisciplinares, tanto do Ensino Médio como Superior, como por exemplo da aplicação de conceitos de Geometria Analítica na implementação do “empurrão” gerado pelos inimigos ao colidir com o jogador, utilizando-se do cálculo de vetores e sua modularização.

TABELA DE CONCEITOS UTILIZADOS E NÃO UTILIZADOS

Nesta seção, será detalhada em formato de tabela, todos os conceitos utilizados e não utilizados durante o desenvolvimento do trabalho. A tabela apresenta também onde tais conceitos foram implementados e uma breve justificativa do seu uso, ressaltando a sua importância no projeto.

Tabela 2. Lista de Conceitos Utilizados e Não Utilizados no Trabalho.

N.	Conceitos	Uso	O quê / Onde & Justificativa sucinta em uma frase
1	Elementares:		
1.1 &	- Classes, objetos. & - Atributos (privados), variáveis e constantes. - Métodos (com e sem retorno).	Sim	- Na maioria dos .h e .cpp, como nas classes nos <i>namespaces</i> Entidades e Animador. - Classes, atributos e métodos são elementos básicos da programação orientada a objetos.
1.2 &	- Métodos (com retorno const e parâmetro const). - Construtores (sem/com parâmetros) e destrutores	Sim	- Na maioria dos .h e .cpp, como nas classes nos <i>namespaces</i> Entidades e Animador. - O uso de constantes evita mudanças equivocadas, construtores são mandatórios para inicializar atributos e destrutores para finalizações como desalocações.
1.3	- Classe Principal.	Sim	- Main.cpp & Jogo.h/.cpp - Uma classe Principal é mais 'purista' em termos de OO, visto que centraliza a lógica geral do jogo em uma classe.
1.4	- Divisão em .h e .cpp.	Sim	- No desenvolvimento como um todo, como nas classes nos <i>namespaces</i> Personagens e Obstáculos. - Permite organizar as classes e afins que compõem o sistema.
2	Relações de:		
2.1	- Associação direcional. & - Associação bidirecional.	Sim	- Em vários dos .h e .cpp, como nas classes nos <i>namespaces</i> Personagens e entre as classes Menu e Gerenciador de Eventos. - A associação bidirecional permite que duas classes utilizem atributos e métodos uma da outra, fortalecendo as relações entre elas. Tanto a bidirecional como a direcional são fundamentais para estabelecer dependência entre classes, melhorando a organização do código.
2.2 &	- Agregação via associação. - Agregação propriamente dita.	Sim	- Em vários dos .h e .cpp, como nas classes nos <i>namespaces</i> Listas e Parallax. - A agregação, tanto via associação quanto a propriamente dita, permite uma relação mais complexa e intimista de dependência entre duas classes.
2.3 &	- Herança elementar. - Herança em vários níveis.	Sim	- Em vários .h e .cpp, como nas classes nos <i>namespaces</i> Entidades e Obstáculos. - A Herança em vários níveis permite a reutilização de métodos e atributos que duas classes possuem em comum. Isso evita repetições e gera uma maior otimização do código.
2.4	- Herança múltipla pertinente.	Não	Conceito NÃO utilizado
3	Ponteiros, generalizações e exceções		
3.1	- Operador this para fins de relacionamento bidirecional	Sim	- Em vários dos .h e .cpp, como nas classes nos <i>namespaces</i> Personagens e entre as classes Menu e Gerenciador de Eventos. - O operador <i>this</i> permite que uma classe se faça conhecida por outra.

3.2	- Alocação de memória (new & delete).	Sim	- Em vários .h e .cpp, como nas classes nos namespaces Parallax e Listas. - A alocação de memória permite a criação de instâncias dinâmicas, ou seja, criadas e deletadas em tempo de execução.
3.3	- Gabaritos/Templates criada/adaptados pelos autores para Listas.	Sim	- Nas classes Lista e Elemento, de <i>namespaces</i> Listas. - Classes templates e gabaritos permitem a sua reutilização em contextos pertinentes.
3.4	- Uso de Tratamento de Exceções (try catch).	Sim	Nos métodos de salvamento das classes de <i>namespace</i> Fases - <i>Try Catch</i> permite o correto tratamento de exceções e erros.
4	Sobrecarga de:		
4.1	- Construtoras e Métodos.	Sim	- Nas classes Gerenciador_Colisoes e Jogador - A sobrecarga de métodos e construtoras permite que instâncias e procedimentos semelhantes sejam diferenciados pelos parâmetros dos métodos.
4.2	- Operadores (2 tipos de operadores pelo menos).	Sim	- Nas classes Menu e Lista - A sobrecarga de operadores permite uma simplificação da chamada de métodos, tornando-os mais intuitivos.
---	Persistência de Objetos (via arquivo de texto ou binário)		
4.3	- Persistência de Objetos.	Sim	- Nas classes Menu_Principal e Menu_Pause e nos <i>namespace</i> Entidades e Fases. - A persistência de objetos permite a recuperação dos atributos dos objetos após o programa ter sido encerrado.
4.4	- Persistência de Relacionamento de Objetos.	Sim	- Nas classes Menu_Principal e Menu_Pause e nos <i>namespace</i> Entidades e Fases. - A persistência de relacionamentos permite a recuperação dos relacionamentos entre os objetos após o programa ter sido encerrado.
5	Virtualidade:		
5.1	- Métodos Virtuais Usuais.	Sim	- Em vários .h e .cpp, como nas classes nos <i>namespaces</i> Personagens e Obstaculos. - Métodos virtuais permitem a especialização de métodos base.
5.2	- Polimorfismo extensivamente à luz do padrão arquitetural (i.e., diagrama de classes assaz genérico) dado.	Sim	- Em vários .h e .cpp, como nas classes nos <i>namespaces</i> Personagens e Obstaculos. - O polimorfismo permite que a classe pai chame métodos especializados de cada uma das suas classes herdadas.
5.3	- Métodos Virtuais Puros / Classes Abstratas à luz do padrão arquitetural.	Sim	- Em vários .h e .cpp, como nas classes nos <i>namespaces</i> Personagens e Entidades. - Métodos virtuais puros são utilizados quando todas as classes herdadas possuem um método em comum, que não possui implementação na classe pai, mas que pode ser acessada por ele de maneira polimórfica.
5.4	- Coesão/Desacoplamento efetiva e intensa inclusive com o apoio de mais de 5 padrões de projeto.	Não	Conceito NÃO utilizado.
6	Organizadores e Estáticos		
6.1	- Espaço de Nomes (Namespace) criada pelos autores.	Sim	- Em vários .h e .cpp, como os <i>namespaces</i> Entidades e Listas. - Namespaces permitem a melhor organização e visualização das partes do projeto.
6.2	- Classes aninhadas (Nested) criada ou adaptadas pelos autores.	Sim	- Em vários .h e .cpp, como nas classes nos <i>namespaces</i> Parallax e Listas. - Classes aninhadas permitem uma melhor organização e utilização de classes intimamente ligadas e que não se comunicam diretamente com as demais classes externas.

6.3	- Atributos estáticos e métodos estáticos.	Sim	- Nas classes de <i>namespaces</i> Gerenciadores e na classe Ente. - Atributos e métodos estáticos permitem que o valor de um atributo ou o funcionamento de um método seja compartilhado com todos os objetos daquela classe.
6.4	- Uso extensivo de constante (const) parâmetro, retorno, método...	Sim	- Em vários .h e .cpp, como nas classes no <i>namespace</i> Animadores. - Métodos constantes garantem que nenhuma mudança será realizada em um dado atributo, mantendo sua integridade.
7	Standard Template Library (STL) e String OO		
7.1	- A classe Pré-definida String ou equivalente. & - Vector e/ou List da STL (p/ objetos ou ponteiros de objetos de classes definidos pelos autores)	Sim	- Em vários .h e .cpp, como nas classes nos <i>namespaces</i> Animadores e Listas. - Vector e List são formas práticas e úteis de agrupar instâncias e variáveis, enquanto String permite a fácil utilização de conjuntos de char.
7.2	- Pilha, Fila, Bifila, Fila de Prioridade, Conjunto, Multi-Conjunto, Mapa OU Multi-Mapa.	Sim	- Em um Map na classe Animação, de <i>namespaces</i> Animadores. - Map é uma forma rápida e prática de agrupar dados que se relacionam em pares.
---	Programação concorrente		
7.3	- Threads (Linhas de Execução) no âmbito da Orientação a Objetos, utilizando Posix, C-Run-Time OU Win32API ou afins.	Não	Conceito NÃO utilizado.
7.4	- Threads (Linhas de Execução) no âmbito da Orientação a Objetos com uso de Mutex, Semáforos, OU Troca de mensagens.	Não	Conceito NÃO utilizado.
8	Biblioteca Gráfica / Visual		
8.1	- Funcionalidades Elementares. & - Funcionalidades Avançadas como: tratamento de colisões, duplo buffer ou outros.	Sim	- No Gerenciador de Colisões, de <i>namespaces</i> Gerenciadores. - Colisões e o uso de uma biblioteca gráfica (nesse caso, a SFML) permitem uma melhor jogabilidade e estética para o jogo.
8.2	- Programação orientada e evento efetiva (com gerenciador apropriado de eventos inclusive, via padrão de projeto Observer) em algum ambiente gráfico. OU - RAD – Rapid Application Development (Objetos gráficos como formulários, botões etc).	Não	Conceito NÃO utilizado.
---	Interdisciplinaridades via utilização de Conceitos de Matemática Contínua e/ou Física.		
8.3	- Ensino Médio Efetivamente.	Sim	- Na classe Inimigo, de <i>namespace</i> Entidades-Personagens, através da Força de atrito. - A força de atrito foi utilizada para desenvolver o “empurrão” que os inimigos exercem sobre o jogador.
8.4	- Ensino Superior Efetivamente.	Sim	- Na classe Inimigo, de <i>namespace</i> Entidades-Personagens, através do cálculo e modularização de vetores. - O cálculo de vetores foi utilizado para tornar mais realista a direção do “empurrão” do inimigo.
9	Engenharia de Software		

9.1	- Compreensão, melhoria e rastreabilidade de cumprimento de requisitos.	Sim	- Por meio da leitura constante do documento de especificação, construção das tabelas de requisitos e conceitos e do desenvolvimento do Diagrama de Classes. - A rastreabilidade do cumprimento de requisitos permite a melhoria constante e estratégica deles, impactando na qualidade do projeto.
9.2	- Diagrama de Classes em UML.	Sim	- O diagrama de Classes foi desenvolvido segundo as solicitações do professor através do software Astah UML. - O Diagrama de Classes permite uma melhor rastreabilidade de requisitos e estruturação das relações entre as classes do projeto.
9.3	- Uso efetivo e intensivo de padrões de projeto GOF, i.e., mais de 5 padrões.	Não	Requisito NÃO utilizado.
9.4	- Testes à luz da Tabela de Requisitos e do Diagrama de Classes.	Sim	A implementação foi feita seguindo as relações estabelecidas no Diagrama de Classes
10 Execução de Projeto			
10.1 &	- Controle de versão de modelos e códigos automatizado (via github). - Uso de alguma forma de cópia de segurança (i.e., backup).	Sim	Foi utilizado o GitHub e um pen-drive. https://github.com/MEG3832/TecProg-Projeto.git Ferramenta fundamental para compartilhamento de código entre os membros da dupla, além de servir como um repositório seguro do trabalho.
10.2	- Reuniões com o professor para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA DO TRABALHO]	Sim	28/10 – Reunião 0 com o professor. 04/11 – Reunião 1 com o professor. 18/11 – Reunião 2 com o professor.
10.3	- Reuniões com monitor da disciplina para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA DO TRABALHO]	Sim	03/11- Reunião com o monitor Felipe Simbalista 10/11 - Reunião com o monitor Felipe Simbalista 14/11 – Participação na Oficina de Salvamento, oferecida pelo PETECO 17/11 - Reunião com o monitor Felipe Simbalista
10.4 &	- Escrita do trabalho e feitura da apresentação - Revisão do trabalho escrito de outra equipe e vice-versa.	Sim	A escrita do trabalho e a feitura da apresentação foi feita de maneira colaborativa entre os membros da dupla (Rachel e Maria Eduarda). A revisão foi feita pela dupla composta pelos alunos Rodrigo Dub Meneghello e Rafael Fulgencio Rosario da Cruz, da turma S71.
Total de conceitos apropriadamente utilizados. <i>(Cada grande tópico vale 10% do total de conceitos. Assim, caso se tenha feito metade de um tópico, então ele valerá 5%.)</i>		85% (oitenta e cinco por cento).	

DISCUSSÃO E CONCLUSÕES

O desenvolvimento de um jogo como trabalho da disciplina se mostrou uma ótima oportunidade para aplicar de forma prática as técnicas e os conceitos de programação orientada a objetos estudados durante o semestre. Ademais, com esse trabalho acadêmico, permitiu-se uma compreensão mais aprofundada sobre organização, estruturação e manutenção de códigos maiores e mais elaborados.

Um ponto de destaque durante o trabalho foi a de modelagem através da aplicação do método simplificado de Engenharia de Software, que tem como cerne o uso de diagramas

UML, que são fundamentais para esquematizar classes e suas relações. Dessa forma, foi possível estruturar o sistema de maneira modular, favorecendo o desacoplamento e a coesão do código posteriormente implementado.

Vale ressaltar também que a realização do projeto colaborou fortemente na consolidação dos conhecimentos a respeito do uso do GitHub. Esta plataforma foi constantemente utilizada durante a elaboração do jogo, sendo usada como repositório e como uma forma de compartilhamento e versionamento dos códigos feitos.

Além disso, durante a realização do projeto, fez-se necessária a divisão de tarefas e o exercício do trabalho em equipe. Respeitando as diferentes habilidades e disponibilidades, cada integrante contribuiu de forma estratégica no desenvolvimento do jogo, com uma complementando o trabalho da outra e vice-versa. Essa colaboração mútua foi essencial para a conclusão do trabalho, resultando na entrega de um jogo funcional e que atende majoritariamente os requisitos propostos.

Por fim, destaca-se a forte ajuda fornecida pelos monitores e pelo PETECO, que dedicaram seu tempo a nós e aos estudantes da disciplina ao longo do desenvolvimento do jogo, esclarecendo dúvidas e nos auxiliando sempre que possível.

DIVISÃO DO TRABALHO

Esta seção apresenta uma tabela mostrando a separação das atividades realizadas durante o trabalho, em que se buscou uma divisão equilibrada entre os membros da dupla.

Tabela 4. Lista de Atividades e Responsáveis.

N.	Atividades	Responsáveis
1	Elementares: AMBOS	
1.1	- Classes, objetos. & - Atributos (privados), variáveis e constantes. & - Métodos (com e sem retorno).	Rachel e Maria Eduarda
1.2	- Métodos (com retorno <i>const</i> e parâmetro <i>const</i>). & - Construtores (sem/com parâmetros) e destrutores	Rachel e Maria Eduarda
1.3	- Classe Principal.	Rachel e Maria Eduarda
1.4	- Divisão em .h e .cpp.	Rachel e Maria Eduarda
2	Relações de: AMBOS	
2.1	- Associação direcional. & - Associação bidirecional.	Rachel e Maria Eduarda
2.2	- Agregação via associação. & - Agregação propriamente dita.	Rachel e Maria Eduarda
2.3	- Herança elementar. & - Herança em vários níveis.	Rachel e Maria Eduarda
2.4	- Herança múltipla.	Não feito
3	Ponteiros, generalizações e exceções: AMBOS	
3.1	- Operador <i>this</i> para fins de relacionamento bidirecional.	Rachel e Maria Eduarda
3.2	- Alocação de memória (<i>new</i> & <i>delete</i>).	Rachel e Maria Eduarda
3.3	- Gabaritos/ <i>Templates</i> criada/adaptados pelos autores para Listas.	Rachel e Maria Eduarda
3.4	- Uso de Tratamento de Exceções (<i>try catch</i>).	Rachel e Maria Eduarda
4	Sobrecarga de: AMBOS	
4.1	- Construtoras e Métodos.	Rachel e Maria Eduarda
4.2	- Operadores (2 tipos de operadores pelo menos)	Rachel e Maria Eduarda
---	Persistência de Objetos (via arquivo de texto ou binário): AMBOS	
4.3	- Persistência de Objetos.	Rachel e Maria Eduarda
4.4	- Persistência de Relacionamento de Objetos.	Rachel e Maria Eduarda
5	Virtualidade: AMBOS	
5.1	- Métodos Virtuais Usuais.	Rachel e Maria Eduarda

5.2	- Polimorfismo.	Rachel e Maria Eduarda
5.3	- Métodos Virtuais Puros / Classes Abstratas.	Rachel e Maria Eduarda
5.4	- Coesão/Desacoplamento efetiva e intensa com o apoio de padrões de projeto (mais de 5 padrões).	Não feito.
6	Organizadores e Estáticos: AMBOS	
6.1	- Espaço de Nomes (<i>Namespace</i>) criada pelos autores.	Rachel e Maria Eduarda
6.2	- Classes aninhadas (<i>Nested</i>) criada pelos autores.	Rachel e Maria Eduarda
6.3	- Atributos estáticos e métodos estáticos.	Rachel e Maria Eduarda
6.4	- Uso extensivo de constante (<i>const</i>) parâmetro, retorno, método...	Rachel e Maria Eduarda
7	Standard Template Library (STL) e String OO: AMBOS	
7.1	- A classe Pré-definida <i>String</i> ou equivalente. & - <i>Vector</i> e/ou <i>List</i> da STL (p/ objetos ou ponteiros de objetos de classes definidos pelos autores)	Rachel e Maria Eduarda
7.2	- Pilha, Fila, Bifila, Fila de Prioridade, Conjunto, Multi-Conjunto, Mapa OU Multi-Mapa.	Rachel e Maria Eduarda
---	Programação concorrente: NÃO FEITO	
7.3	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos, utilizando Posix, C-Run-Time OU Win32API ou afins.	Não feito.
7.4	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos com uso de Mutex, Semáforos, OU Troca de mensagens.	Não feito.
8	Biblioteca Gráfica / Visual: AMBOS	
8.1	- Funcionalidades Elementares. & - Funcionalidades Avançadas como: tratamento de colisões e duplo <i>buffer</i>	Rachel e Maria Eduarda
8.2	- Programação orientada a evento efetiva (com gerenciador apropriado de eventos inclusive, via padrão de projeto <i>Observer</i>) em algum ambiente gráfico. OU - RAD – <i>Rapid Application Development</i> (Objetos gráficos como formulários, botões etc).	Não foi feito.
---	Interdisciplinaridades via uso de Conceitos de Matemática Contínua e/ou Física: AMBOS	
8.3	- Ensino Médio Efetivamente.	Rachel e Maria Eduarda
8.4	- Ensino Superior Efetivamente.	Rachel e Maria Eduarda
9	Engenharia de Software: AMBOS	
9.1	- Compreensão, melhoria e rastreabilidade de cumprimento de requisitos.	Rachel e Maria Eduarda
9.2	- Diagrama de Classes em UML.	Rachel e Maria Eduarda
9.3	- Uso efetivo e intensivo de padrões de projeto GOF, i.e., + de 5 padrões.	Não feito.
9.4	- Testes à luz da Tabela de Requisitos e do Diagrama de Classes.	Rachel e Maria Eduarda
10	Execução de Projeto: AMBOS	
10.1	- Controle de versão de modelos e códigos automatizado (via github). & - <i>Uso de alguma forma de cópia de segurança (i.e., backup).</i>	Rachel e Maria Eduarda
10.2	- Reuniões com o professor para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO A ENTREGA DO TRABALHO]	Rachel e Maria Eduarda
10.3	- Reuniões com monitor da disciplina para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA]	Rachel e Maria Eduarda
10.4	- Escrita do trabalho e feitura da apresentação & - Revisão do trabalho escrito de outra equipe e vice-versa.	Rachel e Maria Eduarda

- Maria Eduarda trabalhou em 100% das atividades, as realizando ou colaborando nelas efetivamente.

- Rachel trabalhou em 100% das atividades, as realizando ou colaborando nelas efetivamente.

AGRADECIMENTOS PROFISSIONAIS

Expressamos nossa gratidão ao monitor Felipe Simbalista pela sua constante dedicação e auxílio a nossa dupla e também as outras equipes, além do PETECO por proporcionar as oficinas que também nos ajudaram no desenvolvimento do trabalho.

REFERÊNCIAS UTILIZADAS NO DESENVOLVIMENTO

- [A] YouTube. “Criando um Jogo em C++ do ZERO”. Disponível em: [Playlist - Criando um Jogo em C++ do ZERO](#). (acessado em 23 de novembro de 2025)
- [B] SFML Team, “Simple and Fast Multimedia Library (SFML).” SFML-dev.org. <https://www.sfm1-dev.org/> (acessado em 23 de novembro de 2025).
- [C] SIMÃO, J. M. Site das Disciplina de Fundamentos de Programação 2, Curitiba – PR, Brasil
<https://pessoal.dainf.ct.utfpr.edu.br/jeansimao/Fundamentos2/LinguagemC++/Fundamentos1-2-LinguagemC++.htm> (acessado em 23 de novembro de 2025).
- [D] CraftPix.net, “2D & 3D Game Assets,” *CraftPix.net*. <https://craftpix.net/> (acessado em 23 de novembro de 2025).
- [E] Itch.io. “Top free Game assets tagged 2D”. Disponível em: <https://itch.io/game-assets/free/tag-2d> (acessado em 23 de novembro de 2025).
- [F] YouTube. “Learn JSON in C++ in 15 Minutes: Complete nlohmann/json Tutorial”. Disponível em: [Learn JSON in C++ in 15 Minutes: Complete nlohmann/json Tutorial](#). (acessado em 23 de novembro de 2025).
- [G] Giovanenero. “JogoPlataforma2D-Jungle”. Disponível em: [GitHub - Giovanenero/JogoPlataforma2D-Jungle](#) (acessado em 23 de novembro de 2025).